Natural Deduction Proof Checker with Scope Management

Fatemeh Abshang

Spring 2024–2025

Abstract

This document presents a comprehensive Python implementation of a natural deduction proof checker for propositional logic with full scope management. The system validates complete proofs by checking each line against natural deduction rules, managing nested scopes for assumption-based reasoning, and ensuring proper application of introduction and elimination rules. The checker supports 15 inference rules including conjunction, disjunction, implication, negation, and classical logic principles.

Contents

1 Introduction **3**
1.1 Natural Deduction Proofs . . . 3
1.2 Scope Management . . . 3
1.3 Project Features . . . 3
1.4 Supported Rules . . . 3

2 Input Format **3**
2.1 Proof File Structure . . . 3
2.2 Example Proof . . . 4

3 Scope Management System **4**
3.1 Scope Data Structure . . . 4
3.2 Scope Operations . . . 5
3.2.1 Scope Stack . . . 5
3.2.2 Accessibility Check . . . 5

4 Key Algorithms **5**
4.1 Parsing Algorithm . . . 5
4.2 Formula Normalization . . . 5
4.3 Operator Splitting . . . 7

5 Rule Validation **7**
5.1 Introduction Rules . . . 7
5.1.1 Implication Introduction (→i) . . . 7
5.1.2 Negation Introduction (¬i) . . . 8

5.1.3 OR Elimination (∨e) . . . 8
5.2 Elimination Rules . . . 9
5.2.1 AND Elimination (∧e1, ∧e2) . . . 9
5.3 Classical Rules . . . 9
5.3.1 Law of Excluded Middle (LEM) . . . 9
5.3.2 Proof by Contradiction (PBC) . . . 10

6 Complete Validation Algorithm **10**

7 Example Proofs **10**
7.1 Example 1: Simple Proof . . . 10
7.2 Example 2: Proof with Scope . . . 11
7.3 Example 3: Invalid Proof . . . 11

8 Complexity Analysis **12**
8.1 Time Complexity . . . 12
8.2 Space Complexity . . . 12

9 Usage Instructions **12**
9.1 Running the Checker . . . 12
9.2 Common Errors . . . 12

10 Limitations and Extensions **12**
10.1 Current Limitations . . . 12
10.2 Potential Extensions . . . 13

11 Conclusion **13**

A Quick Reference **13**
A.1 Rule Summary . . . 13

1 Introduction

1.1 Natural Deduction Proofs

A natural deduction proof is a sequence of formulas where:

- Some formulas are **premises** (given)
- Some formulas are **assumptions** (temporary hypotheses)
- Other formulas are derived using **inference rules**
- The final formula is the **conclusion**

1.2 Scope Management

Scopes allow temporary assumptions:

- `BeginScope`: Start new assumption context
- `EndScope`: Close assumption context
- Formulas in a scope are only valid within that scope
- Outer scopes can access formulas from parent scopes

1.3 Project Features

- Parse proof files with line numbers, formulas, and rules
- Manage nested scopes with accessibility checks
- Validate 15 natural deduction rules
- Check proof correctness line-by-line
- Report invalid deductions with line numbers

1.4 Supported Rules

2 Input Format

2.1 Proof File Structure

```
[line_num]    [formula]    [rule], [refs]
BeginScope
[line_num]    [formula]    [rule], [refs]
...
EndScope
[line_num]    [formula]    [rule], [refs]
```

Components:

Rule	Notation	Type
AND Introduction	∧i	Elimination
AND Elimination	∧e1, ∧e2	Elimination
OR Introduction	∨i1, ∨i2	Introduction
OR Elimination	∨e	Elimination
Implication Intro.	→i	Introduction
Implication Elim.	→e	Elimination
Negation Intro.	¬i	Introduction
Negation Elim.	¬e	Elimination
Double Neg. Elim.	¬¬e	Elimination
Double Neg. Intro.	¬¬i	Introduction
False Elimination	⊥e	Elimination
Modus Tollens	MT	Derived
Proof by Contradiction	PBC	Derived
Law of Excluded Middle	LEM	Classical
Copy	Copy	Structural

Table 1: Complete list of supported rules

- Line number (positive integer)
- Formula (4+ spaces after line number)
- Rule name (4+ spaces after formula)
- References (comma-separated line numbers or ranges)

2.2 Example Proof

```
1    p ^ q    Premise
2    p        ^e1, 1
3    q        ^e2, 1
BeginScope
4    r        Assumption
5    p ^ r    ^i, 2, 4
EndScope
6    r -> (p ^ r)    ->i, 4-5
```

3 Scope Management System

3.1 Scope Data Structure

```python
scope_metadata = {
    0: {  # Root scope
        'start_file_line': 0,
        'end_file_line': None,
        'parent_scope_id': None
    },
    4: {  # Nested scope starting at line 4
        'start_file_line': 3,
```

```
        'end_file_line': 7,
        'parent_scope_id': 0
    }
}
```

Listing 1: Scope metadata

3.2 Scope Operations

3.2.1 Scope Stack

Maintains current nesting level during parsing:

- Push when encountering `BeginScope`
- Pop when encountering `EndScope`
- Track parent-child relationships

3.2.2 Accessibility Check

Algorithm 1 Check Line Accessibility

```
function IsAccessible(currentScope, targetScope, metadata)
    ancestors ← GetAncestors(currentScope, metadata)
    return targetScope ∈ ancestors
end function
function GetAncestors(scopeId, metadata)
    path ← []
    current ← scopeId
    while current ≠ null do
        Insert current at beginning of path
        if current = 0 then
            break
        end if
        current ← metadata[current].parent_scope_id
    end while
    return path
end function
```

Key principle: A line in scope S can reference a line in scope T only if T is an ancestor of S.

4 Key Algorithms

4.1 Parsing Algorithm

4.2 Formula Normalization

Removes unnecessary outer parentheses:

Algorithm 2 Parse Proof File

```
function ReadInputFile
    lines ← ReadFile()
    proofLines ← []
    scopeStack ← [(0, 0)]                                   ▷ (scopeId, parentId)
    scopeMetadata ← {0 : {...}}
    for each line in lines do
        if line = BeginScope then
            newScopeId ← currentFileLine
            parentId ← scopeStack.top().scopeId
            Push (newScopeId, parentId) to scopeStack
            Add metadata for newScopeId
        else if line = EndScope then
            closedScope ← scopeStack.pop()
            Update closedScope end line
        else
            Parse line number, formula, rule
            currentScope ← scopeStack.top()
            Add to proofLines with currentScope
        end if
    end for
    return proofLines, scopeMetadata
end function
```

```python
def norm_frm(formula_str):
    s = formula_str.strip()

    while len(s) >= 2 and s.startswith('(') and s.endswith(')'):
        depth = 0
        can_remove = True

        for i in range(len(s)):
            if s[i] == '(':
                depth += 1
            elif s[i] == ')':
                depth -= 1

            if depth == 0 and i < len(s) - 1:
                can_remove = False
                break

        if can_remove and depth == 0:
            s = s[1:-1].strip()
        else:
            break

    return s
```

Listing 2: Formula normalization

4.3 Operator Splitting

Finds main operator outside parentheses:

```
def split_by_operator(formula, operator):
    idx = find_main_operator(formula, operator)
    if idx == -1:
        return None
    left = formula[:idx].strip()
    right = formula[idx+len(operator):].strip()
    return (left, right)

def find_main_operator(formula, operator):
    depth = 0
    i = 0
    while i < len(formula):
        if formula[i] == '(':
            depth += 1
        elif formula[i] == ')':
            depth -= 1
        elif depth == 0 and formula[i:i+len(operator)] == operator:
            return i
        i += 1
    return -1
```

Listing 3: Split by operator

5 Rule Validation

5.1 Introduction Rules

5.1.1 Implication Introduction (→i)

Rule:

$$\frac{\text{Assume } A \quad \vdots \quad B}{A \to B} \to \text{i}$$

Requirements:

1. Lines m–n form a scope
2. Line m is an assumption (A)
3. Line n is the conclusion (B)
4. Scope parent is current line's scope
5. Result: $A \to B$

```python
if rule_name == '→i':
    if len(par_ref) != 1 or not isinstance(par_ref[0], tuple):
        return False

    start_line, end_line = par_ref[0]
    ass_line = get_line_by_num(all_lines, start_line)
    con_line = get_line_by_num(all_lines, end_line)

    if not ass_line or not con_line:
        return False

    if ass_line['rule_string'] != 'Assumption':
        return False

    target_scope = find_enclosing_scope(all_lines,
                                        scope_metadata,
                                        start_line, end_line)
    if target_scope is None:
        return False

    if scope_metadata[target_scope]['parent_scope_id']
       != current_line_scope_id:
        return False

    expected = f"{norm_frm(ass_line['formula'])} →
                 {norm_frm(con_line['formula'])}"
    return norm_frm(formula) == norm_frm(expected)
```

Listing 4: Implication introduction validation

5.1.2 Negation Introduction (¬i)

Rule:

$$\frac{\text{Assume } A \quad \vdots \quad \bot}{\neg A}\neg\text{i}$$

Similar to →i but conclusion must be ⊥

5.1.3 OR Elimination (∨e)

Rule:

$$\frac{A \vee B \quad [A]\vdots C \quad [B]\vdots C}{C} \vee \text{e}$$

Requirements:

1. Main disjunction: $A \vee B$
2. First scope: Assume A, derive C
3. Second scope: Assume B, derive C
4. Both scopes are children of current scope

5. Result: C

5.2 Elimination Rules

5.2.1 AND Elimination (∧e1, ∧e2)

Rules:

$$\frac{A \wedge B}{A} \wedge \text{e1} \qquad \frac{A \wedge B}{B} \wedge \text{e2}$$

Validation:

1. Check referenced line is accessible
2. Verify it's a conjunction (∧)
3. Extract left (e1) or right (e2) conjunct
4. Compare with current line's formula

5.3 Classical Rules

5.3.1 Law of Excluded Middle (LEM)

Rule:

$$\frac{}{A \vee \neg A}\text{LEM}$$

Validation:

- No references required
- Formula must be $P \vee \neg P$ for some P

```python
if rule_name == 'LEM':
    if len(par_ref) != 0:
        return False

    formula_norm = norm_frm(formula)
    parts = split_by_operator(formula_norm, OR)
    if parts is None:
        return False

    left_part, right_part = parts
    left_part = norm_frm(left_part)
    right_part = norm_frm(right_part)

    if right_part.startswith(NOT):
        return norm_frm(right_part[len(NOT):]) == left_part

    return False
```

Listing 5: LEM validation

5.3.2 Proof by Contradiction (PBC)

Rule:

$$\frac{\text{Assume } \neg A \quad \vdots \quad \bot}{A}\text{PBC}$$

6 Complete Validation Algorithm

Algorithm 3 Check Line Validity

```
function CheckLineValidity(line, allLines, scopeMetadata)
    rule ← line.rule_string
    formula ← line.formula
    scope ← line.scope_id
    if rule ∈ {Premise, Assumption} then
        return true
    end if
    ruleName, refs ← ParseRule(rule)
      ruleName ∧i
    Verify two line references
    Check both accessible
    Build A ∧ B and compare ∧e1
    Verify one line reference
    Check is conjunction
    Extract left part and compare →i
    Verify scope range
    Check assumption and conclusion
    Verify scope structure
    Build A → B and compare ...
    ...
end function
```

7 Example Proofs

7.1 Example 1: Simple Proof

Proof file:

```
1    p ^ q        Premise
2    p            ^e1, 1
3    q            ^e2, 1
4    q ^ p        ^i, 3, 2
```

Validation:

1. Line 1: Premise (always valid)
2. Line 2: ∧e1 on line 1 ✓

3. Line 3: ∧e2 on line 1 ✓
4. Line 4: ∧i on lines 3, 2 ✓

Output: `Valid Deduction`

7.2 Example 2: Proof with Scope

Proof file:

```
1    p            Premise
BeginScope
2    q            Assumption
3    p ^ q        ^i, 1, 2
EndScope
4    q -> (p ^ q)    ->i, 2-3
```

Scope structure:

- Scope 0: Lines 1, 4
- Scope 3: Lines 2, 3 (parent: 0)

Validation:

1. Line 1: Premise ✓
2. Line 2: Assumption ✓
3. Line 3: ∧i, line 1 accessible from scope 3 ✓
4. Line 4: →i on scope 2–3, parent is scope 0 ✓

Output: `Valid Deduction`

7.3 Example 3: Invalid Proof

Proof file:

```
BeginScope
1    p            Assumption
EndScope
2    p            Copy, 1
```

Problem: Line 2 (scope 0) tries to access line 1 (scope 1), but scope 1 is not ancestor of scope 0.

Output: `Invalid deduction at Line 2`

8 Complexity Analysis

8.1 Time Complexity

- **Parsing**: $O(n \cdot m)$ where n is lines, m is avg formula length
- **Scope management**: $O(n \cdot d)$ where d is max nesting depth
- **Validation**: $O(n \cdot k)$ where k is avg validation cost per rule
- **Total**: $O(n \cdot (m + d + k))$

8.2 Space Complexity

- **Proof storage**: $O(n \cdot m)$
- **Scope metadata**: $O(s)$ where s is number of scopes
- **Total**: $O(n \cdot m + s)$

9 Usage Instructions

9.1 Running the Checker

```
python proof_checker.py

Please enter a file name:
proof.txt

# Output:
Valid Deduction
# or
Invalid deduction at Line X
```

9.2 Common Errors

1. **Scope mismatch**: Referenced line not accessible
2. **Wrong rule**: Formula doesn't match rule pattern
3. **Missing scope**: →i, ¬i without proper scope
4. **Invalid reference**: Line number doesn't exist

10 Limitations and Extensions

10.1 Current Limitations

1. No automatic proof search

2. No proof optimization
3. Limited error messages
4. No proof transformation

10.2 Potential Extensions

1. **Proof assistant**: Suggest next valid rules
2. **Proof search**: Automatic theorem proving
3. **Proof compression**: Remove redundant steps
4. **LaTeX export**: Generate formatted proofs
5. **Interactive mode**: Step-by-step validation

11 Conclusion

This natural deduction proof checker implements a complete validation system for propositional logic proofs with scope management. The system correctly handles nested assumptions, validates all standard natural deduction rules, and provides clear feedback on proof validity. The scope management system enables proper handling of assumption-discharge patterns essential for implication and negation introduction rules.

A Quick Reference

A.1 Rule Summary

Rule	**Pattern**	**References**
$\wedge$i	$A, B \Rightarrow A \wedge B$	2 lines
$\wedge$e1, $\wedge$e2	$A \wedge B \Rightarrow A$ or B	1 line
$\vee$i1, $\vee$i2	$A \Rightarrow A \vee B$ or $B \vee A$	1 line
$\vee$e	$A \vee B, [A]C, [B]C \Rightarrow C$	1 line + 2 scopes
$\rightarrow$i	$[A]B \Rightarrow A \rightarrow B$	1 scope
$\rightarrow$e	$A \rightarrow B, A \Rightarrow B$	2 lines
$\neg$i	$[A]\bot \Rightarrow \neg A$	1 scope
$\neg$e	$A, \neg A \Rightarrow \bot$	2 lines
$\neg\neg$e	$\neg\neg A \Rightarrow A$	1 line
$\neg\neg$i	$A \Rightarrow \neg\neg A$	1 line
$\bot$e	$\bot \Rightarrow A$	1 line
MT	$A \rightarrow B, \neg B \Rightarrow \neg A$	2 lines
PBC	$[\neg A]\bot \Rightarrow A$	1 scope
LEM	$\Rightarrow A \vee \neg A$	none
Copy	$A \Rightarrow A$	1 line

Table 2: Quick reference for all rules